Day 20: Deploy a Windows Wazuh Agent

Theoretical Walkthrough of the Agent Deployment Procedure

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	22 September 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Install and enroll the Wazuh agent on a Windows lab VM; verify service status locally and confirm the endpoint is visible and active in Wazuh. (Tools: Wazuh Agent, Windows, PowerShell/CMD)
Status of this Report	**Theoretical walkthrough of the deployment and enrollment procedure. Despite four days of sustained effort (18-22 September) and multiple deployment approaches, a working Wazuh manager could not be brought online due to insufficient hardware resources on the available lab machines, so this task could not be completed practically. This report demonstrates the correct procedure in full detail and will be re-submitted with real command output and screenshots once a manager is available.**

Table of Contents

Table of Contents 2
Key Terms 3
Important Note on This Report 3
Executive Summary 4
1. Objective 5
2. Prerequisite: A Running Wazuh Manager, and Why It Is Not Yet Available 5
3. Planned Procedure 6
3.1 Obtain the Wazuh agent MSI installer 6
3.2 Install and enroll the agent (silent install, manager address set at install time) 6
3.3 Start the agent service 6
3.4 Verify the service locally 7
3.5 Confirm enrollment from the Wazuh Dashboard 7
4. What Will Be Recorded as Evidence Once Executed 7
5. Conclusion 7

Key Terms

- **Wazuh Agent:** A lightweight program installed on an endpoint that collects security-relevant data (Windows Event Logs, file integrity changes, registry changes, running processes) and forwards it to the Wazuh manager.
- **msiexec:** The Windows Installer command-line utility used to install .msi packages, including the Wazuh agent installer, silently and with configuration parameters passed directly at install time.
- **WazuhSvc:** The Windows service name the Wazuh agent registers under once installed; controlled with NET START/STOP or PowerShell's Start-Service/Stop-Service, and checked with sc query or Get-Service.
- **WAZUH_MANAGER:** An install-time parameter that tells the Windows agent the IP address or hostname of the Wazuh manager to enroll and communicate with, written into the agent's local configuration automatically.
- **Agent Management / Agent List:** The section of the Wazuh Dashboard listing every enrolled agent and its status (active, disconnected, never connected); used to confirm the Windows endpoint enrolled successfully.
- **PowerShell / CMD:** The two Windows command-line environments the agent installer, service start command, and status checks can be run from; both are shown below since either is acceptable for the Day 20 task.
- **Windows Firewall:** Windows' built-in outbound/inbound traffic filter; a common cause of enrollment failure if it blocks the agent's outbound connection to the manager's ports (1514/1515).

Important Note on This Report

This is a theoretical walkthrough, not a record of a completed deployment. No Windows agent was installed or enrolled in this session, because no Wazuh manager is currently running to enroll against. The reason, and the four days of work behind it, are explained in Section 2 below.

Executive Summary

Day 20 asks for a Wazuh agent to be installed and enrolled on a Windows lab VM, with its service status verified locally and its presence confirmed as active from the Wazuh side. Like Day 19's Linux agent, this depends entirely on a running Wazuh manager to enroll against, which is not currently available.

Between 18 and 22 September, sustained effort went into getting a Wazuh manager running: six separate installation attempts of the all-in-one Wazuh stack inside a VMware-virtualized Ubuntu Server VM, using both the full installer and a targeted indexer-only approach, all of which failed at the same point (the wazuh-indexer service failing to start after installing cleanly), root-caused to JVM heap auto-sizing causing memory and CPU exhaustion under the hypervisor on the available 6-8GB hardware, corroborated by kernel-level soft-lockup warnings and systemd timeout logs captured during the attempts.

Following that diagnosis, a corrective migration to a native, dual-boot Ubuntu Server installation (removing the virtualization layer entirely) was begun on the only machine with adequate free space, a shared family desktop used daily by another household member. The preparation phase (partial data backup, freeing and verifying 100GB of disk space) was completed, but because the full contents of that shared machine's drive could not be completely backed up in the time available, proceeding with an irreversible disk partitioning and OS install on a shared, not-fully-backed-up machine was assessed as too great a risk to take unilaterally, and the migration was paused rather than completed.

This report documents the correct, verified procedure for Windows agent deployment and enrollment, covering both CMD and PowerShell, so the task's understanding is demonstrated and the exact steps are ready to execute once a manager exists.

1. Objective

Install the Wazuh agent MSI package on the Windows lab VM, configure it at install time to enroll with the Wazuh manager, start the WazuhSvc service, verify its status locally via PowerShell/CMD, and confirm from the Wazuh Dashboard that the endpoint appears with active status.

2. Prerequisite: A Running Wazuh Manager, and Why It Is Not Yet Available

As with the Linux agent, enrollment requires a reachable manager IP address (represented below as <MANAGER_IP>). Over four days of effort, that manager could not be brought online, for the following documented reasons:

- Attempts 1-6 (18-21 September): the Wazuh all-in-one installer was run repeatedly inside a VMware Workstation VM (Ubuntu Server 24.04, 6GB RAM, 2 vCPU) on an 8GB-RAM host. The wazuh-indexer package installed successfully every time, but the service consistently failed to start, timing out after consuming several minutes of real CPU time without becoming ready.

- Diagnostic evidence (a systemd start-operation timeout logged after 5+ minutes of active CPU consumption, and kernel soft-lockup warnings showing a CPU core unresponsive for 20+ seconds) pointed to a specific root cause: OpenSearch, which the Wazuh indexer runs on, auto-sizes its Java heap based on available system RAM, and on a 6GB VM sharing an 8GB host with Windows and VMware itself, this created severe memory pressure and CPU scheduling contention rather than a simple slowdown.

- 22 September: based on that diagnosis, work began on a corrective fix, migrating off virtualization entirely onto a native, dual-boot Ubuntu Server installation. This required a different physical machine than the original lab laptop, since a dual-boot install partitions and repartitions a disk at the operating-system level, an operation that is very low-risk when done carefully but is not fully reversible if interrupted. The only machine with adequate free space for this was a shared family desktop, used daily by another household member for his own ongoing work, not a personal or disposable test machine.

  Before touching any partition, all existing data on the target drive was backed up as a precaution; however, the full contents of that drive could not be copied elsewhere in the time available, meaning a complete, verified backup of everything on the machine could not be guaranteed. Proceeding with an irreversible disk-level change to a shared machine, on data that could not be fully backed up, without another person's direct sign-off on that risk, was assessed as an unacceptable risk to take unilaterally, and the decision was made to pause the migration rather than proceed. This is treated here as a deliberate risk-management decision, consistent with standard change-control practice of not applying an irreversible change to a shared system without a complete rollback path, rather than a shortcut.

The steps below therefore assume a manager is reachable at a known IP address, represented as <MANAGER_IP> throughout, and will be executed exactly as written once that manager exists.

3. Planned Procedure

3.1 Obtain the Wazuh agent MSI installer

The Windows agent ships as a standard .msi installer package (wazuh-agent-4.14.7-1.msi), downloaded from Wazuh's official package repository onto the Windows lab VM ahead of installation. Unlike the Linux agent, there is no repository/apt-style setup step required on Windows; the single downloaded file is self-contained.

3.2 Install and enroll the agent (silent install, manager address set at install time)

The installer is run from the command line rather than double-clicked, so it can be run silently (no click-through wizard) and so the manager's address can be supplied as a parameter in the same step, avoiding any manual configuration file editing afterward.

Using CMD:

```
msiexec.exe /i wazuh-agent-4.14.7-1.msi /q WAZUH_MANAGER="<MANAGER_IP>"
```

Using PowerShell (equivalent command, run from the folder containing the installer):

```
msiexec.exe /i .\wazuh-agent-4.14.7-1.msi /q WAZUH_MANAGER="<MANAGER_IP>"
```

/i tells msiexec to install (as opposed to uninstall or repair) the named package. /q runs the installation quietly, with no on-screen wizard or prompts, which is standard for scripted/documented deployments. WAZUH_MANAGER="<MANAGER_IP>" is read by the installer and written directly into the agent's local configuration file, stored by default under C:\Program Files (x86)\ossec-agent\ossec.conf, so the agent already knows which manager to contact the moment installation finishes.

3.3 Start the agent service

As on Linux, installing the package does not automatically start the Windows service; it must be started explicitly.

Using CMD:

```
NET START WazuhSvc
```

Using PowerShell:

```
Start-Service wazuhsvc
```

Both commands do the same thing through different interfaces: they tell the Windows Service Control Manager to launch the WazuhSvc service now. The Wazuh installer also configures the service to start automatically on future reboots by default, so this manual start step is mainly needed for this first run.

3.4 Verify the service locally

Using CMD:

```
sc query WazuhSvc
```

Using PowerShell:

```
Get-Service wazuhsvc
```

A healthy result shows the service STATE as RUNNING (sc query) or Status as Running (Get-Service). This confirms the agent process itself launched correctly on the endpoint; like the Linux agent, it does not yet confirm the manager has accepted the enrollment, which is the separate check in Section 3.5.

If the service fails to start or immediately stops, the plan is to check the agent's own local log file rather than treat the failure as unexplained, consistent with the evidence-first troubleshooting approach already used successfully during the earlier server-side installation attempts:

```
Get-Content 'C:\Program Files (x86)\ossec-agent\ossec.log' -Tail 50
```

Common causes a log review would be expected to surface include: an unreachable or mistyped WAZUH_MANAGER address, Windows Firewall blocking the outbound connection to the manager's ports (1514 for agent-manager traffic, 1515 for enrollment), or the two machines being on different, unreachable network segments.

3.5 Confirm enrollment from the Wazuh Dashboard

The service running locally is necessary but not sufficient; the Day 20 requirement is that the endpoint is visible and active on the Wazuh side. This is confirmed by logging into the Wazuh Dashboard, navigating to the Agents / Agent Management view, and locating the endpoint by the Windows VM's hostname. A successful enrollment shows the agent listed with Active status and a recently-updated Last Keep Alive timestamp. This is the specific screen that will be captured as a screenshot for the completed version of this report.

4. What Will Be Recorded as Evidence Once Executed

- Terminal/PowerShell output of the msiexec install command completing without error.
- Output of Get-Service wazuhsvc (or sc query WazuhSvc) showing RUNNING.
- A screenshot of the Wazuh Dashboard's Agent Management view showing the Windows endpoint listed with Active status.
- Any troubleshooting actually needed and the fix applied, if enrollment does not succeed on the first attempt (for example, adding a Windows Firewall outbound rule for ports 1514/1515).

5. Conclusion

Windows agent deployment was not practically performed in this session, and this report does not claim otherwise. Across four days, multiple documented approaches (repeated VM-based

installation attempts, and the beginning of a dual-boot migration) were made to bring a Wazuh manager online, constrained by the same underlying issue described in Day 19: insufficient hardware resources on the available lab machines for the virtualization-heavy approaches, and, for the resource-appropriate dual-boot alternative, the responsible decision not to complete an irreversible disk-level change on a shared, not-fully-backed-up household machine without another person's direct sign-off. The procedure above is technically correct and verified against Wazuh's own official documentation for both CMD and PowerShell, and demonstrates the understanding the Day 20 task is assessing. It will be executed for real, with genuine output and a genuine dashboard screenshot, as soon as a Wazuh manager is available, directly following on from Day 19's Linux agent deployment once that manager exists.